

BOGOSORT

DATA STRUCTURE – **PROF. VISOTHBOTI SIN**

PRESENTED BY:

- 1.SOPHIA BELLE RAMOS (2025107)
- 2.THUL VUTHEASAK (2025293)
- 3.NGORN SIHASIN (2025296)
- 4.SIM VIRKAPIKUN(2025360)
- 5.SAM OEUN ROZANEL (2025490)

GROUP 7



TABLE OF CONTENT

I. Introduction

II. How It Works

III. Characteristics

IV. Why BogoSort?

V. Time Complexity

VI. Java Implementation

VII. Testing Methodology & Results

VIII. Comparison

IX. Real-world Application

X. Conclusion and Finding

Bubble sort



**Cocktail
Shaker sort**



Selection sort



Quicksort

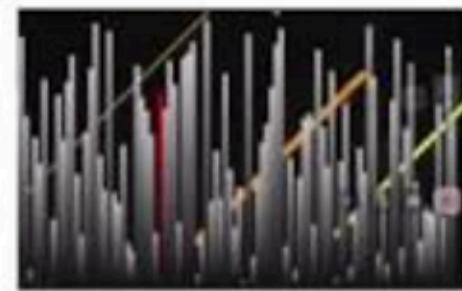


Bogo sort

INTRODUCTION

Selection Sort
Bubble Sort
Insertion Sort
Merge Sort
Quick Sort
Heap Sort
Counting Sort
Radix Sort
Bucket Sort

Bogo Sort



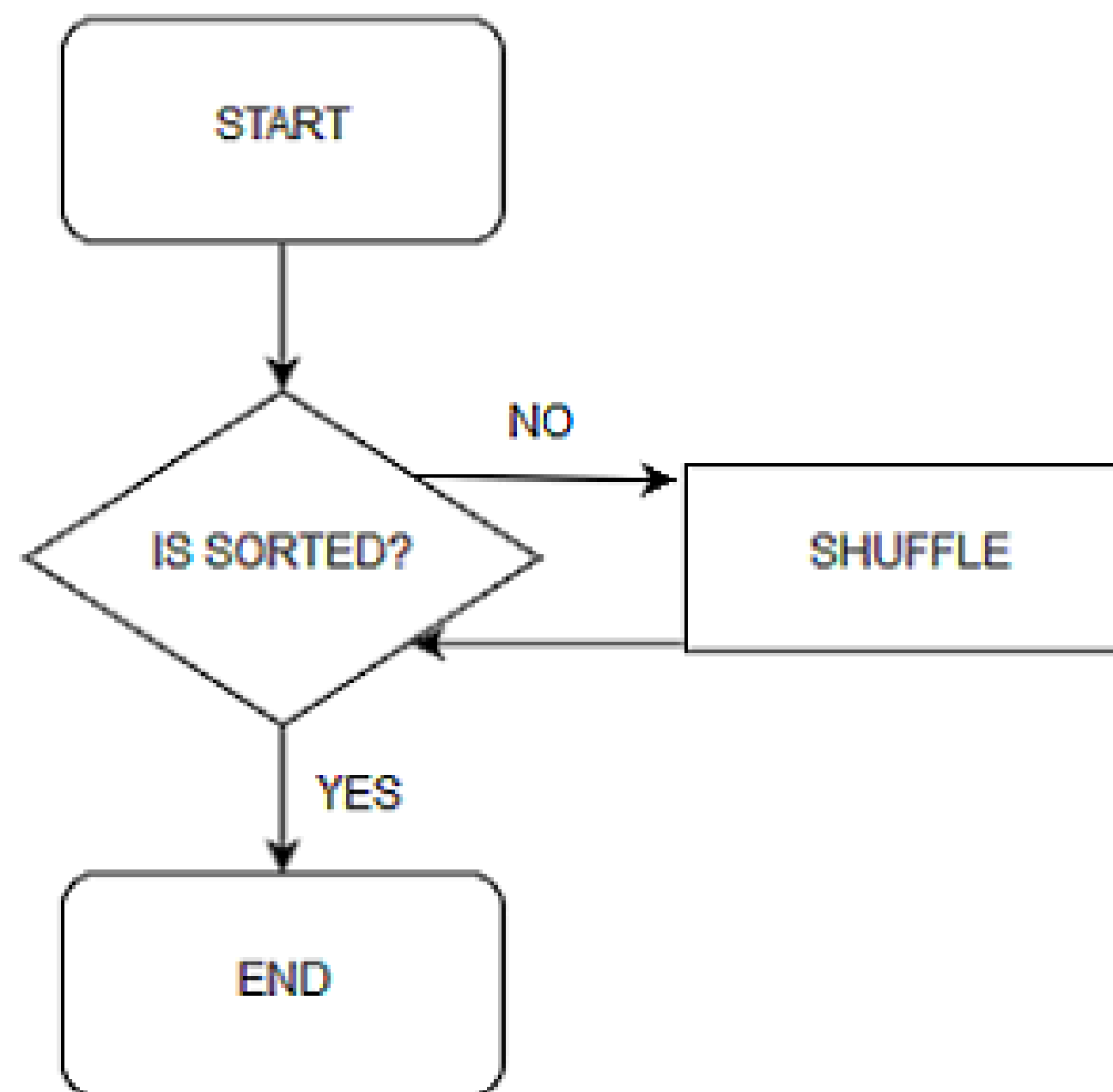
WHAT IS BOGO SORT?

THE SIGMA CHAD OF SORTING ALGORITHM?

“BOGO SORT” OR “STUPID SORT” OR “PERMUTATION
SORT” OR “SHOTGUN SORT”

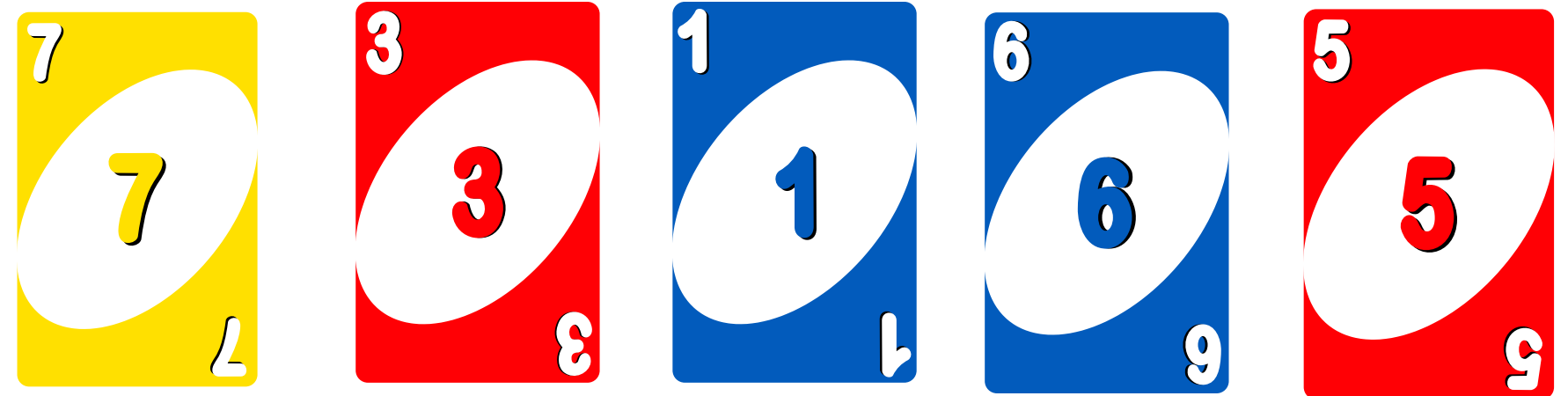
IS KNOWN/ RIDICULED AS THE MOST INEFFICIENT
SORTING ALGORITHM THAT EVER EXISTED.

HOW DOES IT WORK?

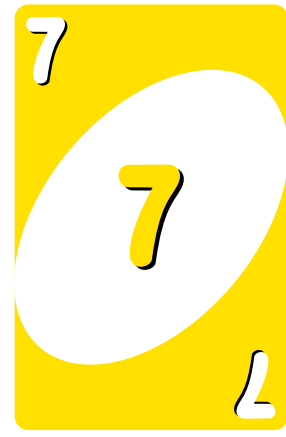


```
BOGOSORT(array A of size n):  
    WHILE isSorted(A) = FALSE DO  
        shuffle(A)  
        attempts ← attempts + 1  
    END WHILE  
  
isSorted(array A of size n):  
    FOR i ← 0 TO n - 2 DO  
        comparisons ← comparisons + 1  
        IF A[i] > A[i+1] THEN  
            RETURN FALSE  
        END IF  
    END FOR  
    RETURN TRUE  
  
SHUFFLE(array A of size n):  
    FOR i ← n-1 DOWN TO 1 DO  
        j ← random integer in [0, i]  
        SWAP A[i] AND A[j]  
        swaps ← swaps + 1  
    END FOR
```

HOW DOES IT WORK?





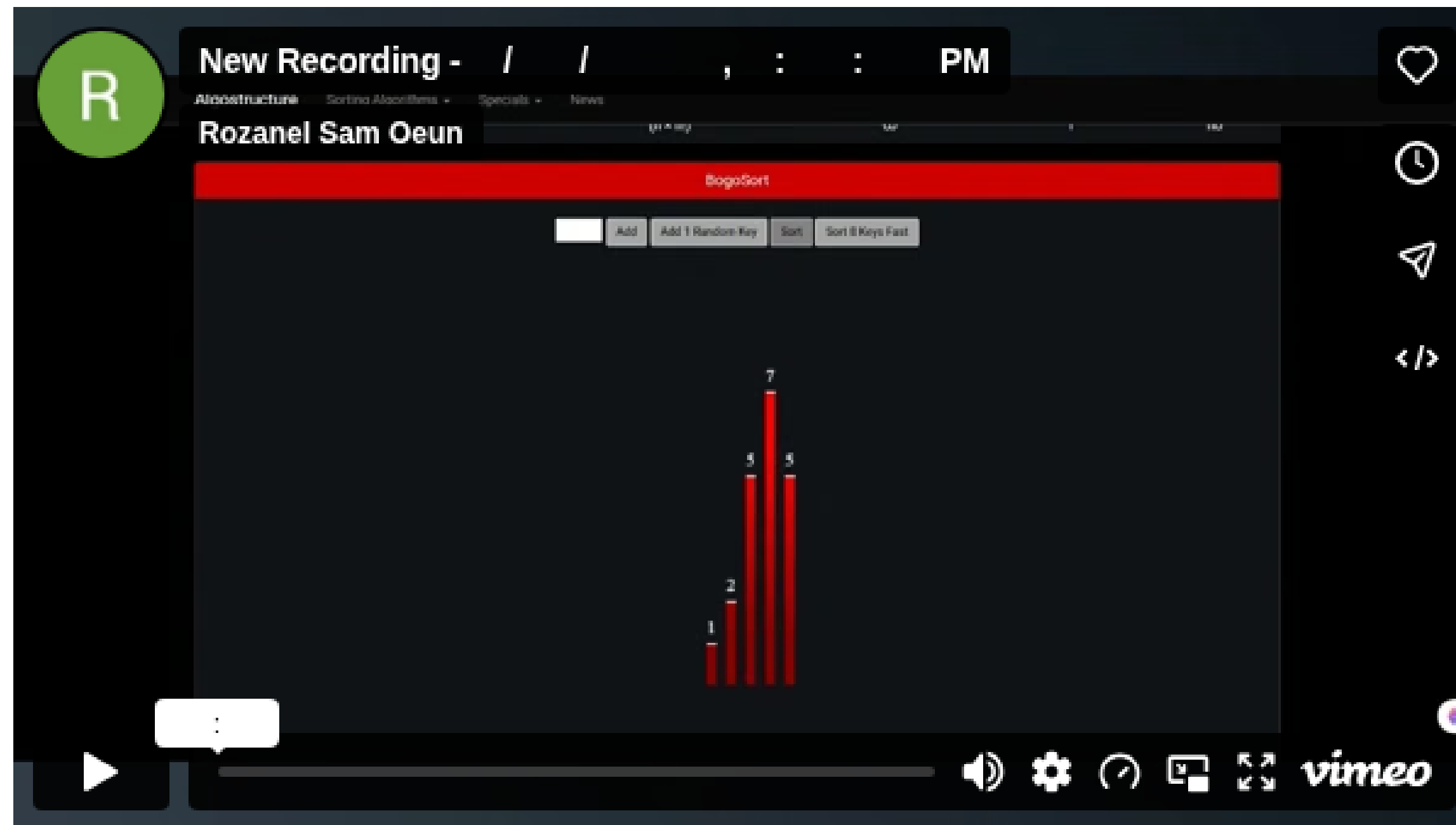




HOW DOES IT WORK?



EXAMPLE



CHARACTERISTIC



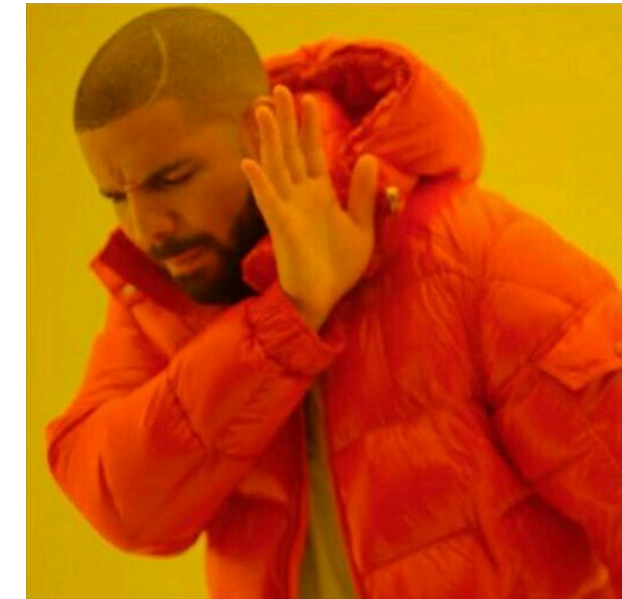
Be a reliable sorting algorithm



let gamble with luck

“Theoretically speaking, has the potential to sort in $O(1)$ ”

“has the potential to defeat quick sort”



Finish the almost sorted array

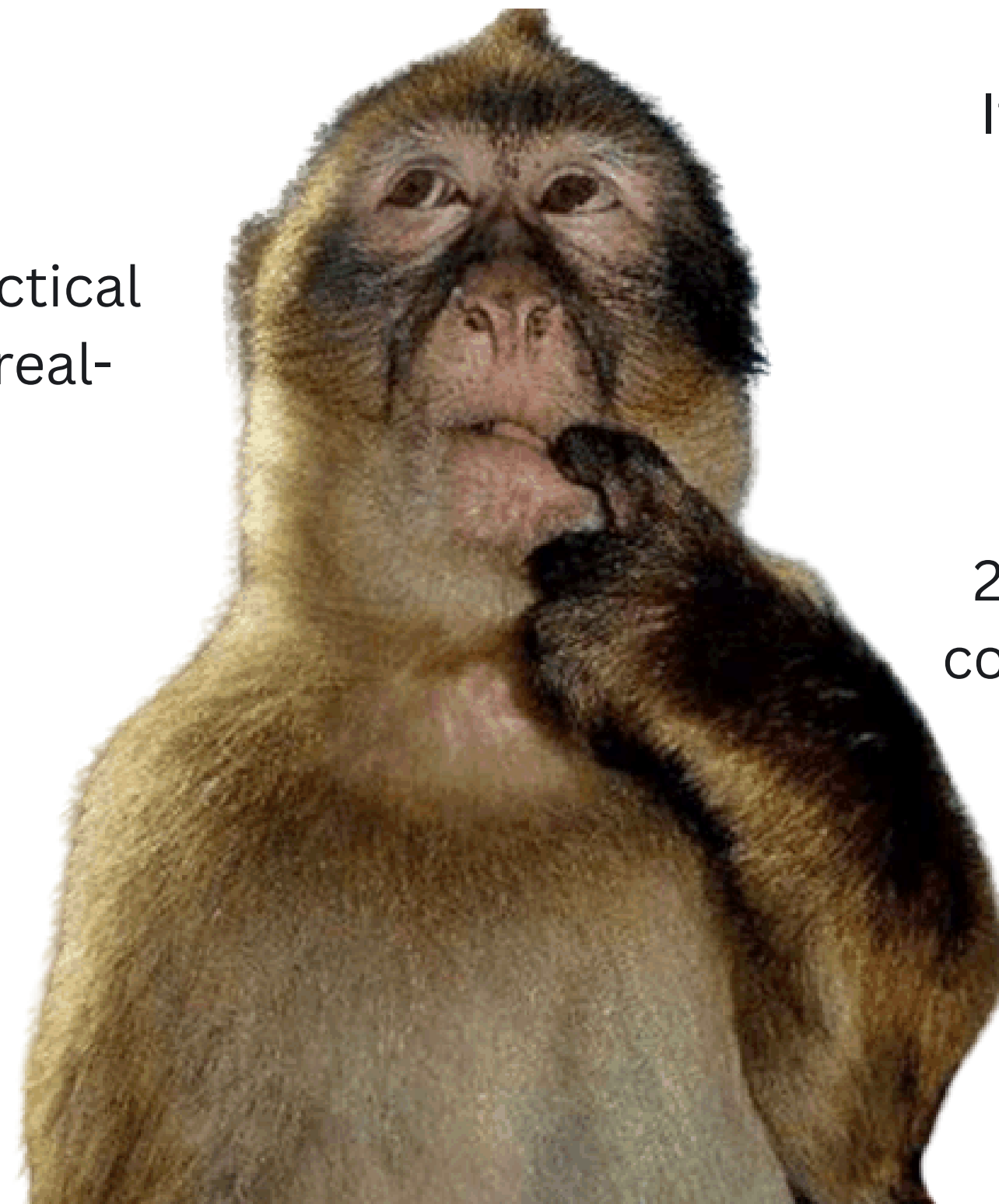


reshuffle the entire thing

“Its time complexity potentially outlive humanity”

WHY WE CHOSE BOGO?

Although Bogo Sort is impractical to implement or apply in a real-world system

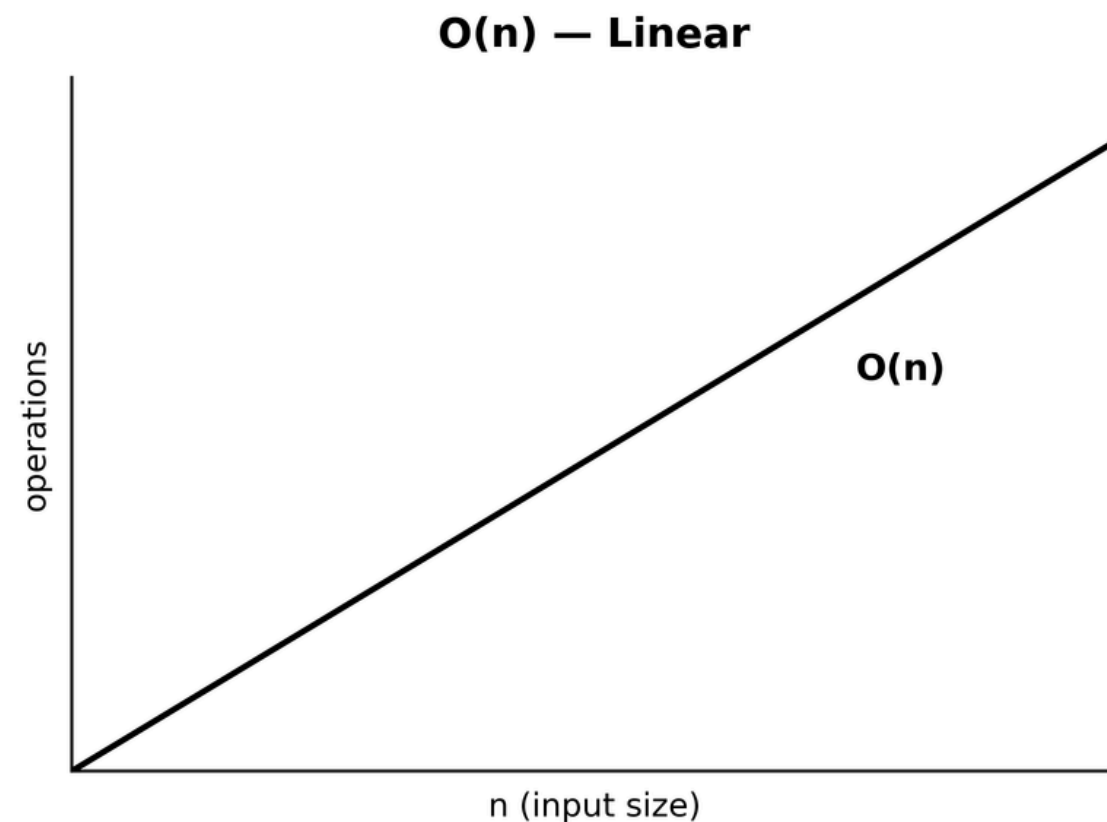


It can be used as a quintessential example that illustrates :

1. what a poor algorithm design looks like
2. highlights why factors like time complexity and algorithm structure are important to write efficient code.

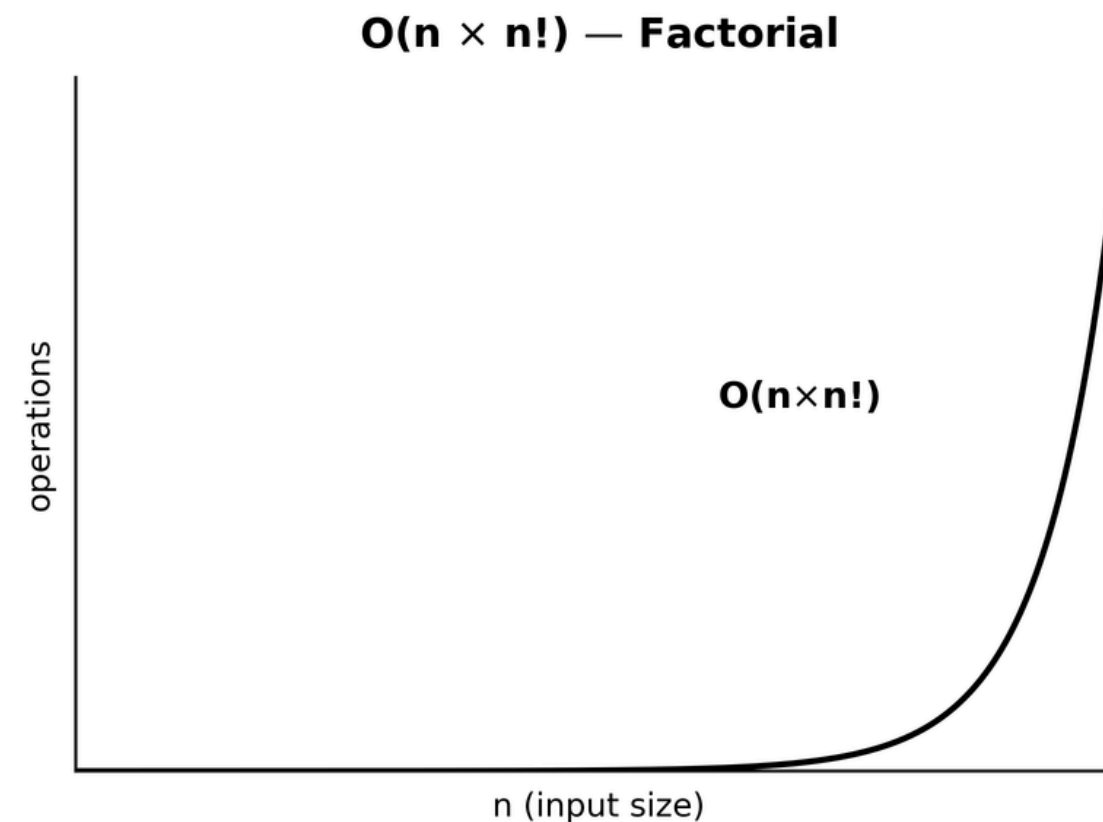
TIME COMPLEXITY

3 main cases, including Best case, Average case, and worst case.



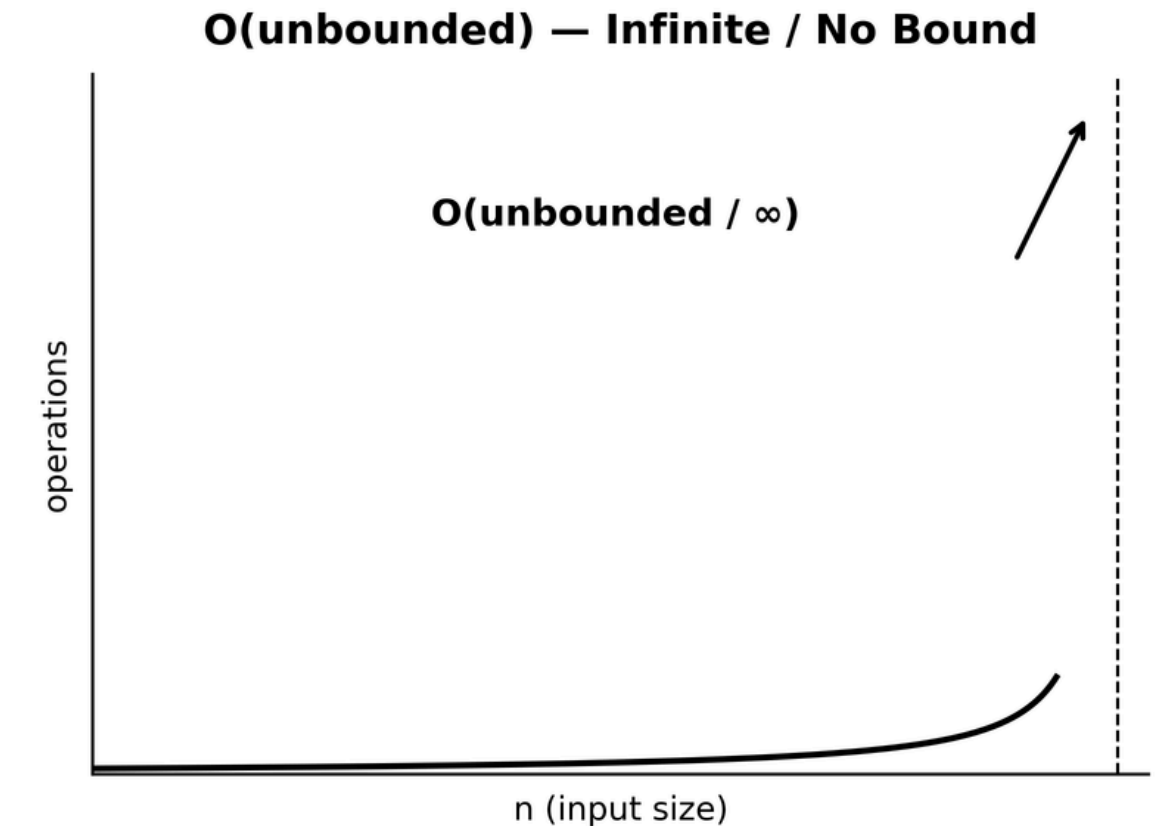
Best case

- Array is already sorted
- One linear scan to verify



Average case

- $n!$ possible permutations
- Random shuffles until array is sorted
- each check takes n times



Worst case

- No guarantee of finding the sorted order
- Could shuffle forever non-stop

JAVA IMPLEMENTATION

The BogoSORT Java program is structured into 5 key components:

01

Array Initialization

Generates n random integers (1–100) using Java's Random class. Dataset size controls how painful the wait will be.

02

isSorted()

Linear scan comparing $\text{arr}[i] > \text{arr}[i+1]$. Increments comparisons counter each check. Returns true only if fully sorted.

03

shuffle()

Fisher-Yates algorithm — picks random index j for each position i and swaps. Each shuffle = n individual swaps.

04

bogoSort()

Core loop: shuffle → check → repeat until sorted. Tracks attempts, comparisons, swaps, and runtime via `nanoTime()`.

05

Performance Output

Prints metrics to console. Saves `performance_report.txt` for all 4 algorithms.



**Finish the
almost
complete
sorted list**

**Reshuffle
the
entire thing
again**

JAVA IMPLEMENTATION – KEY METHODS

isSorted() – Sorted Validation

```
public static boolean isSorted(int[] arr) { 1 usage
    for (int i = 0; i < arr.length - 1; i++) {
        comparisons++;
        if (arr[i] > arr[i + 1]) return false;
    }
    return true;
}
```

How it works:

- Loops through array once ($O(n)$)
- Each comparison increments the counter
- Stops immediately on first out-of-order pair
- Best case: already sorted = 1 scan = $O(n)$

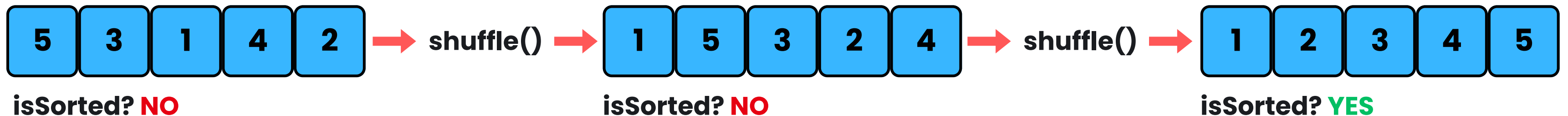
shuffle() – Fisher-Yates algorithm

```
public static void shuffle(int[] arr) { 1 usage
    for (int i = arr.length - 1; i >= 1; i--) {
        int ranIndex = random.nextInt(bound: i + 1);
        int temp = arr[ranIndex];
        arr[ranIndex] = arr[i];
        arr[i] = temp;
        swaps++;
    }
}
```

How it works:

- Iterates from end to start (Fisher-Yates)
- Each element swapped with random earlier position
- Guarantees uniform random permutation
- 1 shuffle = n swaps (array length)

EXAMPLE:



Performance: attempts=3 | comparisons=10 | swaps=15 | runtime= ... ms

JAVA IMPLEMENTATION – CORE & METRICS

bogoSort() – Core Algorithm

```
public static long[] sort(int[] arr) { 1 usage
    comparisons = 0;
    swaps = 0;
    long attempts = 0;
    long startTime = System.nanoTime();

    while (!isSorted(arr)) {
        shuffle(arr);
        attempts++;
    }

    long endTime = System.nanoTime();
    double timeMs = (endTime - startTime) / 1_000_000.0;
    return new long[]{attempts, comparisons, swaps, (long)(timeMs * 1000)};
}
```

=== Sorting Algorithm Performance Report ===
Algorithm under study: Bogo Sort
Compared against: Bubble Sort, Selection Sort, Insertion Sort
Original: [89, 33, 58, 28, 54]

---- Dataset: small (n = 5) ----

Bogo Sort	Attempts: 84	Comparisons: 156	Swaps: 336	Runtime: 0.167 ms
Bubble Sort	Comparisons: 10	Swaps/Movements: 7	Runtime: 0.004 ms	
Selection Sort	Comparisons: 10	Swaps/Movements: 4	Runtime: 0.002 ms	
Insertion Sort	Comparisons: 11	Swaps/Movements: 7	Runtime: 0.003 ms	

Metrics Tracked



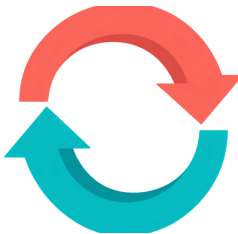
Attempts (Shuffles)

How many times the full array was randomly rearranged before landing sorted. Unique to BogoSort only.



Comparisons

Every time `arr[i] > arr[i+1]` is evaluated inside `isSorted()`. Shared metric with all 4 algorithms.



Swaps

Each shuffle performs exactly `n` swaps. Total = `attempts × n`. BogoSort does far more swaps than others.



Runtime (ms)

Measured with `System.nanoTime()` for precision. Converted to milliseconds for readability.

BogoSorted: [28, 33, 54, 58, 89]
Bubble Sorted: [28, 33, 54, 58, 89]
Selection Sorted: [28, 33, 54, 58, 89]
Insertion Sorted: [28, 33, 54, 58, 89]

TESTING METHODOLOGY

SMALL DATASET



$n = 5$

Empirically tested
(5 trials, averaged)

MEDIUM DATASET



$n = 1,000$

Theoretical only
(BogoSort cannot run)

LARGE DATASET



$n = 1,000,000$

Theoretical only
(BogoSort cannot run)

WHY BOGOSORT CANNOT BE TESTED ON LARGE DATASETS

Average case: $O(n \times n!)$

$n = 1,000 \rightarrow$ Expected shuffles $\approx 1,000 \times 1,000!$ (a number with ~2,500 digits)

$n = 1,000,000 \rightarrow$ Expected shuffles $\approx$ unbounded — no computer could finish this in any practical timeframe



TESTING RESULTS — SMALL DATASET (N = 5)

01

```
---- Dataset: small (n = 5) ----
Bogo Sort | Attempts: 17 | Comparisons: 31 | Swaps: 68 | Runtime: 0.032 ms
Bubble Sort | Comparisons: 10 | Swaps/Movements: 9 | Runtime: 0.002 ms
Selection Sort | Comparisons: 10 | Swaps/Movements: 4 | Runtime: 0.004 ms
Insertion Sort | Comparisons: 13 | Swaps/Movements: 9 | Runtime: 0.002 ms
```

02

```
---- Dataset: small (n = 5) ----
Bogo Sort | Attempts: 284 | Comparisons: 492 | Swaps: 1136 | Runtime: 0.481 ms
Bubble Sort | Comparisons: 10 | Swaps/Movements: 6 | Runtime: 0.003 ms
Selection Sort | Comparisons: 10 | Swaps/Movements: 4 | Runtime: 0.004 ms
Insertion Sort | Comparisons: 10 | Swaps/Movements: 6 | Runtime: 0.003 ms
```

03

```
---- Dataset: small (n = 5) ----
Bogo Sort | Attempts: 152 | Comparisons: 261 | Swaps: 608 | Runtime: 0.286 ms
Bubble Sort | Comparisons: 10 | Swaps/Movements: 10 | Runtime: 0.003 ms
Selection Sort | Comparisons: 10 | Swaps/Movements: 4 | Runtime: 0.004 ms
Insertion Sort | Comparisons: 14 | Swaps/Movements: 10 | Runtime: 0.002 ms
```

04

```
---- Dataset: small (n = 5) ----
Bogo Sort | Attempts: 46 | Comparisons: 77 | Swaps: 184 | Runtime: 0.109 ms
Bubble Sort | Comparisons: 10 | Swaps/Movements: 5 | Runtime: 0.004 ms
Selection Sort | Comparisons: 10 | Swaps/Movements: 4 | Runtime: 0.004 ms
Insertion Sort | Comparisons: 9 | Swaps/Movements: 5 | Runtime: 0.002 ms
```

05

```
---- Dataset: small (n = 5) ----
Bogo Sort | Attempts: 113 | Comparisons: 201 | Swaps: 452 | Runtime: 0.248 ms
Bubble Sort | Comparisons: 10 | Swaps/Movements: 7 | Runtime: 0.004 ms
Selection Sort | Comparisons: 10 | Swaps/Movements: 4 | Runtime: 0.003 ms
Insertion Sort | Comparisons: 11 | Swaps/Movements: 7 | Runtime: 0.002 ms
```

KEY OBSERVATIONS

- BogoSort used 122 attempts on average — but Trial 1 needed only 17, while Trial 2 needed 284. Pure luck.
- All 3 basic sorts finish in <0.01ms with predictable, fixed step counts every single time.
- BogoSort's 490 swaps vs Bubble Sort's 7 swaps — same array, wildly different work done.
- At n=5, even BogoSort finishes fast — the real disaster starts at n=10+.

WHY 5 TRIALS?

- BogoSort is **non-deterministic** — each run produces a completely different number of shuffles due to **randomness**.
- Running multiple trials and averaging gives a more representative result than a single run.

Averaged Results — 5 Trials

Algorithm	Attempts	Comparisons	Swaps	Runtime (ms)
BogoSort	122	212	490	0.231
Bubble Sort	—	10	7	0.003
Selection Sort	—	10	4	0.004
Insertion Sort	—	11	7	0.002

TESTING RESULTS — MEDIUM & LARGE DATASET

Medium Dataset (n = 1,000) — Theoretical

Algorithm	Comparisons	Swaps	Runtime (ms)
BogoSort	unbounded	unbounded	Never finishes
Bubble Sort	499,500	240,224	6.208 ms
Selection Sort	499,500	999	3.754 ms
Insertion Sort	241,223	240,224	3.566 ms

Large Dataset (n = 1,000,000) — Theoretical

Algorithm	Comparisons	Swaps	Runtime (ms)
BogoSort	unbounded	unbounded	Mathematically ∞
Bubble Sort	499,999,500,000	247,508,340,376	2,768,187 ms (~33.6 min)
Selection Sort	499,999,500,000	999,999	1,175,231 ms (~10.8 min)
Insertion Sort	247,509,340,375	247,508,340,376	536,254 ms (~6.1 min)

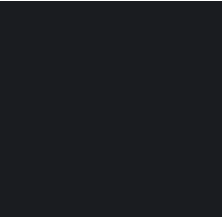
THE TAKEAWAY

- Even the slowest real algorithm (Bubble Sort) finishes n=1,000,000 in 33.6 minutes.
- BogoSort has no projected finish time — it could run forever.
- The 3 basic sorts take milliseconds at n=1,000. BogoSort: never.
- **This confirms:** BogoSort is strictly for education, not production.





COMPARING BOGOSORT TO BUBBLE SORT, SELECTION SORT AND INSERTION SORT



FEATURE COMPARISON CHART

Feature	Bogosort	Bubble Sort	Selection Sort	Insertion Sort
Strategy	Random Permutation	Adjacent Exchange	Subarray Minimum Selection	Incremental Insertion
Best Case	O(n)	O(n) (Optimized)	O(n^2)	O(n)
Average Case	O((n + 1)!)	O(n^2)	O(n^2)	O(n^2)
Worst Case	Unbounded (∞)	O(n^2)	O(n^2)	O(n^2)
Space	O(1)	O(1)	O(1)	O(1)
Stable?	No	Yes	No	Yes
Deterministic?	No	Yes	Yes	Yes

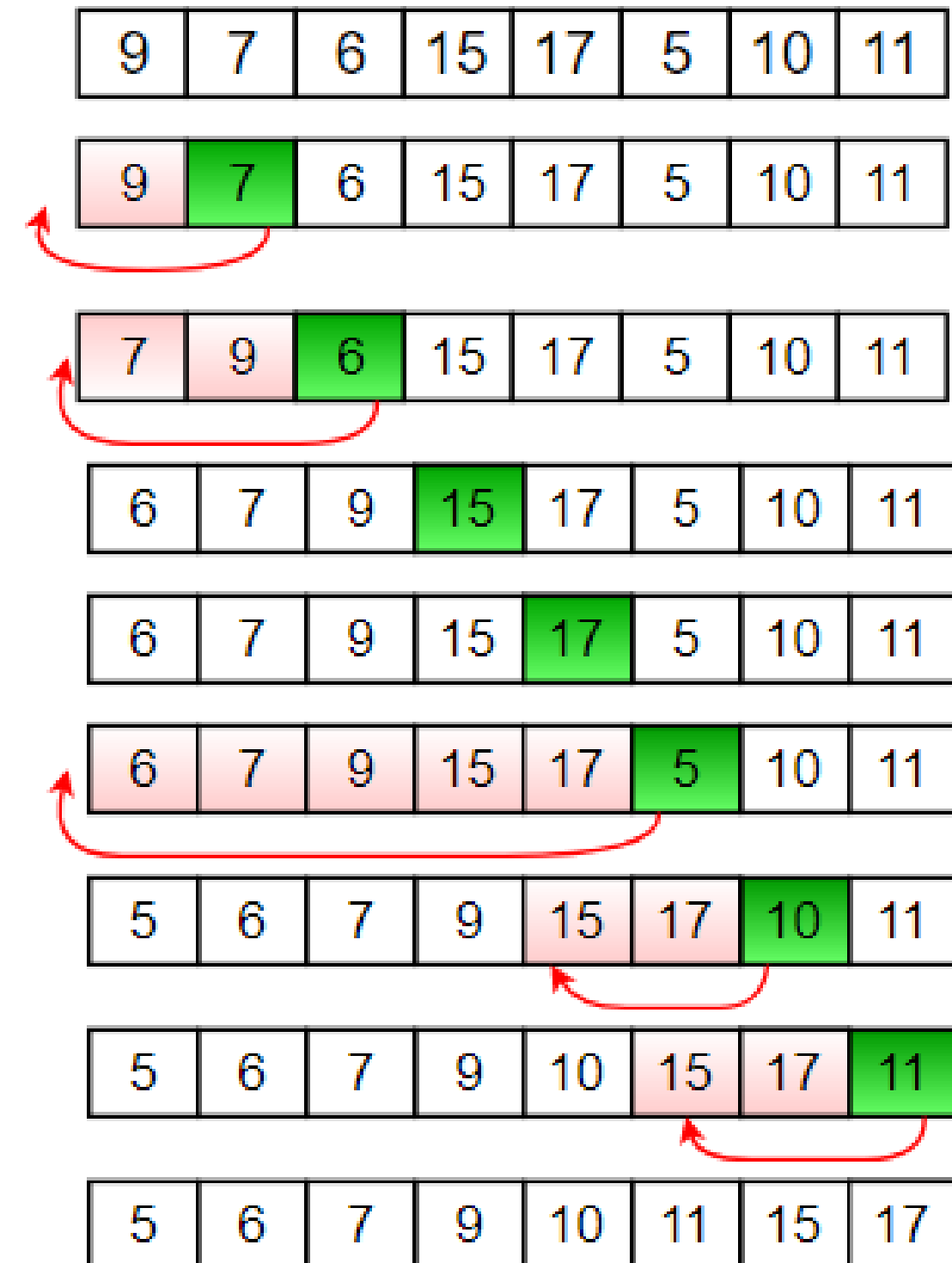
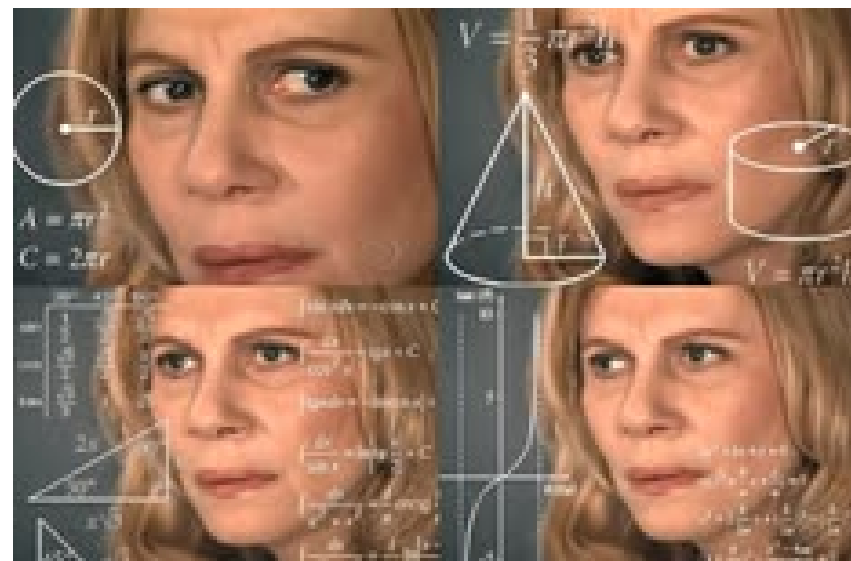
BOGOSORT VS INSERTION SORT

- Adaptability: Insertion Sort resolves nearly-sorted data in near-linear $O(n)$ time; Bogosort treats nearly-sorted arrays identically to randomized ones.
- Practical Utility: Insertion Sort minimizes shifting overhead for small/sorted sets; Bogosort requires identical expected permutations regardless of initial order.

Insertion sort



Bogosort



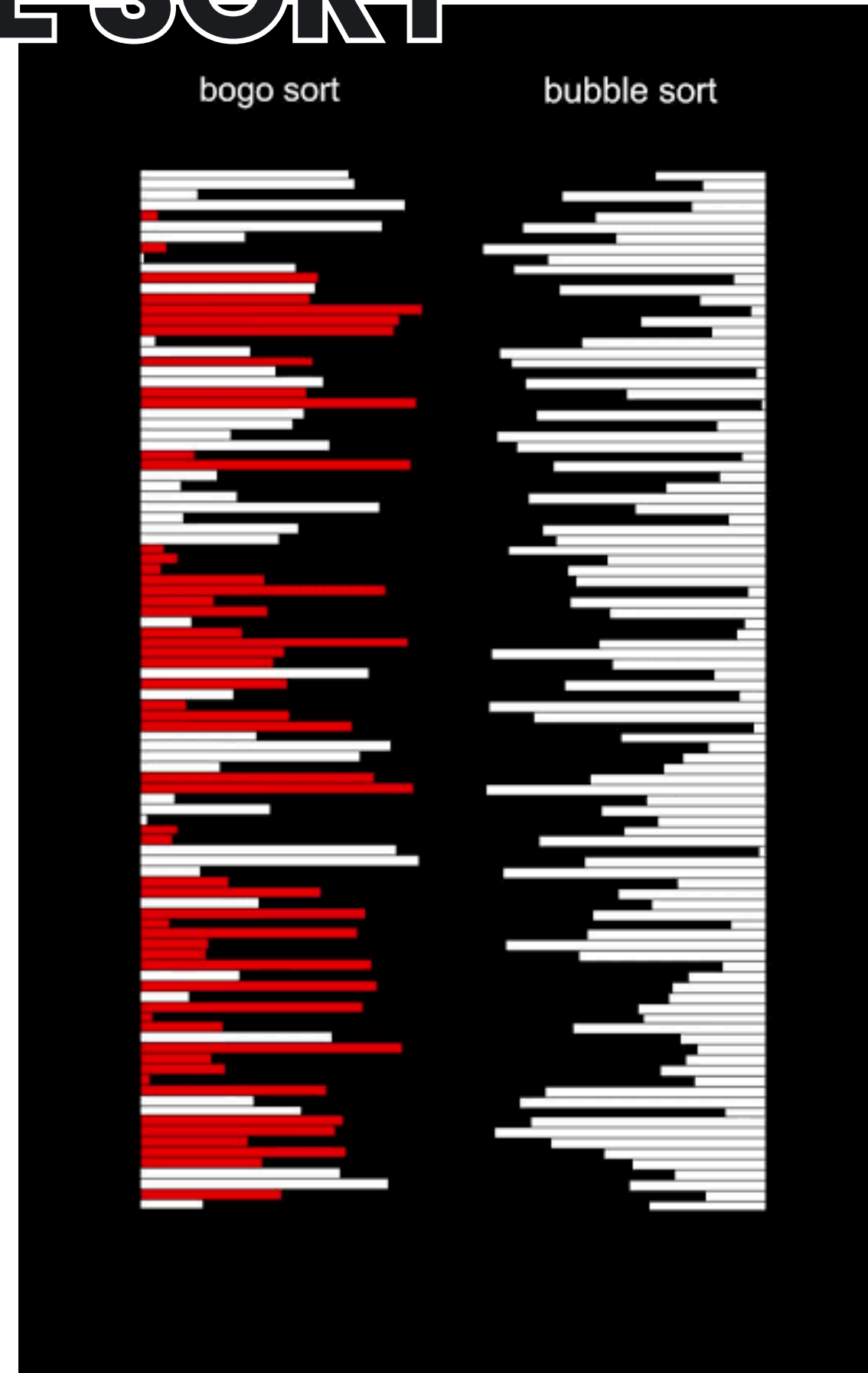
BOGOSORT VS SELECTION SORT

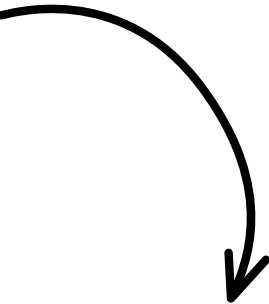


- Deterministic Efficiency: Selection Sort performs a predictable number of comparisons ($\approx n^2/2$) and at most $n - 1$ swaps regardless of data order.
- Write Minimization: Selection Sort limits memory writes; Bogosort averages a staggering $n! \cdot n$ operations for a dataset of size n .

BOGOSORT VS BUBBLE SORT

- Structural Convergence: Bubble Sort guarantees the trailing k elements are in their final positions after pass k ; Bogosort provides zero incremental progress.
- Memoryless Mechanics: Each Bogosort shuffle is entirely independent, whereas Bubble Sort systematically narrows unsorted boundaries.





**WHEN TO OR
WHEN NOT TO
USE BOGOSORT!**

When to use	When not to use
• Education: Serves as a classic counterexample to demonstrate factorial (n!) growth and probability theory. • Testing & Fuzzing: Generates random permutations to test (is_sorted()) validators and OS timeout handlers. • Quantum Physics: Featured in thought experiments like Quantum Bogosort to explain Many-Worlds theory.	• General Processing: Unusable due to factorial runtime ($O((n+1)!)$); 15 items require $\approx 1.3 \times 10^{12}$ shuffles. • Real-Time Systems: Has an unbounded worst case, introducing severe latency risks. • Hardware Constraints: Excessive writes ($n! \cdot n$) rapidly destroy Flash and EEPROM memory. • Nearly Sorted Data: Instantly destroys pre-existing order instead of adapting like Insertion Sort.

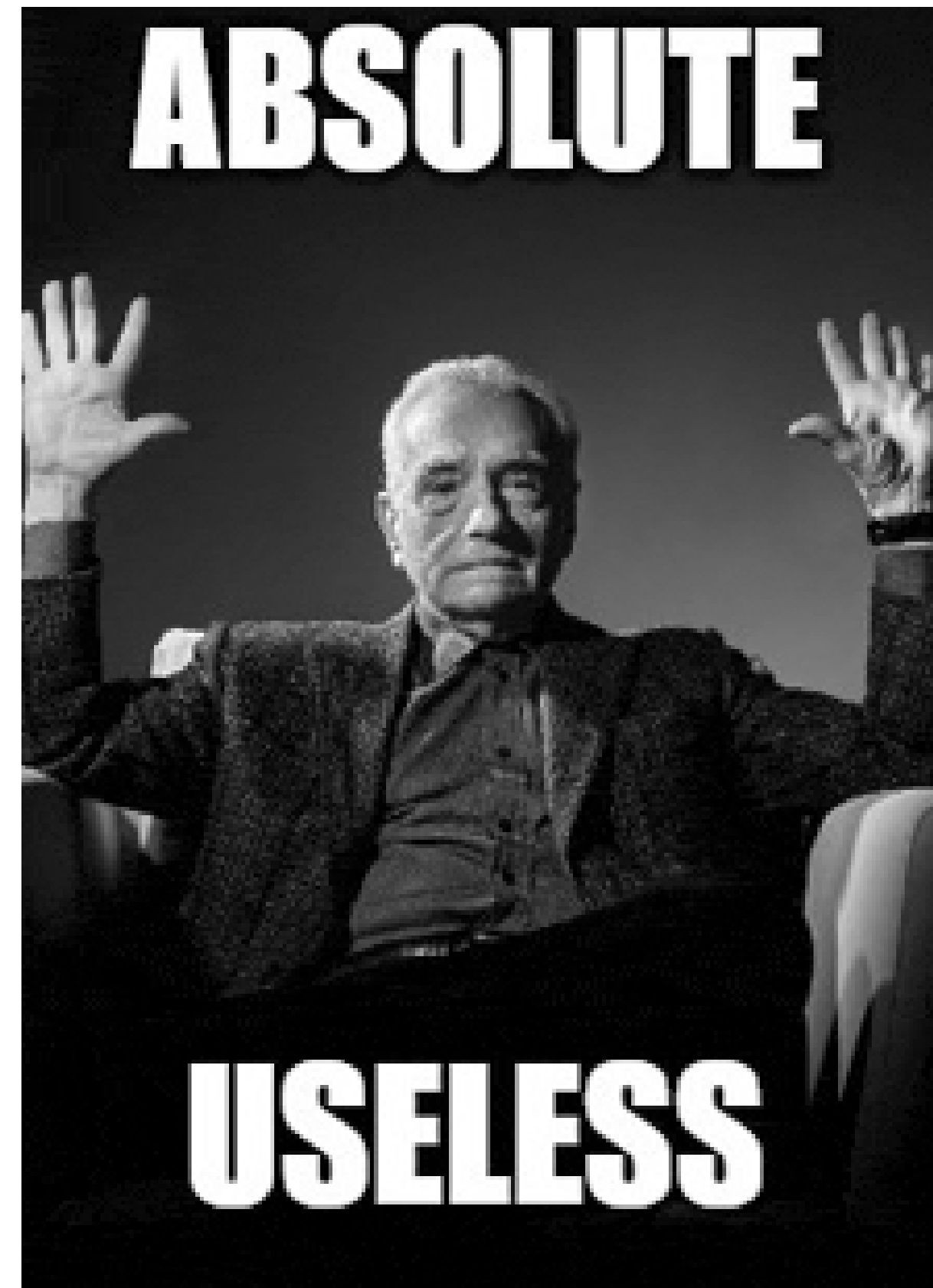
REAL- WORLD APPLICATIONS

1: NO PRACTICAL USES

Due to bogosort being inherently ineffective (time complexity of $O(n \times n!)$), it is never used inside any production related environments

2: EDUCATIONAL PURPOSES

Bogosort still remains used in computer science education and academic research. Researchers use it to showcase flaws in Las Vegas algorithms. It is also used to showcase what **not** to do in algorithm design.



FINDINGS & CONCLUSION



FINDINGS

- **Performance:**

Bogosort is only possible with smaller datasets, and is extremely unpredictable ; results will vary.

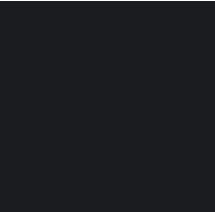
- **Computation Limits:**

Bogosort is impossible at medium ($n=1000$) to large ($n=1,000,000$) datasets

- **Efficiency:**

Compared to other sorting algorithms, BogoSORT relies on pure randomized permutations. It is entirely depended on probability, thus making it unfit for any real-world uses.





CONCLUSION